
Module 1 Fault Code Diagnostic AI

Source Collection: driverdiagnostics

What Collection We Need

- driverdiagnostics contains fault codes, vehicle ID, tenant ID, and telemetry signals per vehicle scan

What Time Interval We Need Data

- On click triggered when the user clicks a fault code on a vehicle, no scheduled interval needed

What Queries We Hit

GET /vehicles/{vehicle_id}/faults returns fault list with severity. Reads from our local DB (no source DB hit). Returns cached diagnostics if already analyzed, falls back to raw staged faults if not yet run:

```
db.fault_vehicles.find_one(
  { "vehicleId": "<vehicle_id>" },
  sort=[("staged_at", -1)]
)
db.diagnostics_output.find({ "source_id": "<source_id>" }, { "_id": 0 })
```

GET /vehicles/faults/diagnose?vehicle_id=&code=&ecu=&desc= triggered on click. Checks KB first, returns instantly if found. If not in KB, runs LLM enrichment once, saves to KB permanently, then returns the full diagnostic:

```
db.knowledge_base.find_one({ "code": "<code>" })
# on KB miss only: LLM runs once, saves result to KB, then:
db.knowledge_base.find_one({ "code": "<code>" })
```

What Infrastructure We Need

- Python / FastAPI service
 - MongoDB (local) for storing our processed data
 - Read-only access to source MongoDB cluster
-

Where We Deploy It

- Separate GitHub repo: **driverbook-fault-diagnostic**
 - Deployed as a standalone API service
-

Where Our Data Gets Stored

- **knowledge_base** fault code definitions, grows permanently as new codes are encountered
 - **fault_vehicles** staged copies of source documents that contain fault codes
 - **diagnostics_output** processed diagnostic results returned to the client app
 - **unknown_faults** codes not yet in knowledge base, enriched and promoted automatically
-

Module 2 Fuel Monitoring Predictive Engine

Source Collection: vehicle-diagnostics-history

What Collection We Need

- vehicle-diagnostics-history time-series records containing fuel level, speed, RPM, coolant temp per vehicle with timestamps
-

What Time Interval We Need Data

- Full history at 10-minute intervals source data already arrives at roughly 10-minute intervals per vehicle
-

What Queries We Hit

Full collection fetch (all records, no filter, batched at 1000):

```
collection.find({}).batch_size(1000)
```

Latest record per vehicle (API on vehicle open):

```
collection.find(
  { "tenantId": "<tenant_id>", "vehicleId": "<vehicle_id>" },
  projection={ "_id": 0, "fuelLevel_pct": 1, "fuel_time_to_empty_hr": 1,
               "fuel_level_alert": 1, "fuel_theft_prediction": 1,
               "vehicle_theft_cumulative_count": 1, ... }
).sort("timestamp", -1).limit(1)
```

Fuel history for a vehicle (last 500 records):

```
collection.find(
  { "tenantId": "<tenant_id>", "vehicleId": "<vehicle_id>" },
  projection={ "_id": 0, "timestamp": 1, "fuelLevel_pct": 1,
               "fuel_theft_prediction": 1 }
).sort("timestamp", 1).limit(500)
```

What Infrastructure We Need

- Python data pipeline (orchestrate.py runs the full processing chain)
- FastAPI backend serving processed results
- MongoDB (local) vehicle_db for source data, fleet_db for processed output
- XGBoost model for fuel theft prediction

Where We Deploy It

- Separate GitHub repo: **driverbook-fuel-monitor**
- Deployed as a standalone API service, independent of Module 1

Where Our Data Gets Stored

- **fleet_db.fuel_latest_features** latest processed record per vehicle, upserted after every pipeline run (keyed on vehicleId)
- **Output includes:** fuel level alert, time to empty, theft prediction flag, theft probability, cumulative theft count, fuel leak flag, idle fuel waste flag, driver aggressiveness score, fleet health score